ARGO CD GITOPS › MY LEARNING NOTES › SESSION 02

First GitOps Application

Topic: UI + CLI + Declarative Create, Sync Loop, Delete | Chapter 2 | 2026-09-12

The GitOps Idea, Concretely

A GitOps application takes **everything from Git** — every YAML/manifest that creates an object must live in a Git repo, not be applied ad hoc. Argo CD is what pulls those files from Git and applies them to the cluster on our behalf.

We'll start with one simple nginx application (Deployment first, then a Service), then make it more complex later.

Every Argo CD Application Needs Exactly 3 Things

1. **Source of truth (Git)** — where Argo CD pulls the object files from.
2. **Target (cluster/namespace)** — where those objects get applied.
3. **Sync policy** — Manual (you click Sync) or Automatic (applied the moment Argo CD detects drift).

Part 1 — Creating the App via the UI

Step 1: Connect the Git Repository

`Settings → Repositories → Connect Repo`

Field	Value
Connection method	HTTPS (also supports SSH)
Type	git
Name	a label for this repo connection
Project	keep `default` for now
Repository URL	the Git URL where the manifest files live
Credentials	not needed for a public repo; private needs username/password or token

Step 2: Create the Application

`Applications → New App`

Field	Value
Application Name	e.g. `nginx-app`
Project	`default`
Sync Policy	`Manual` (chosen first) vs. `Automatic`
Source	connected repo + Revision (branch) + Path (folder inside the repo)
Destination	Cluster URL (in-cluster here) + Namespace

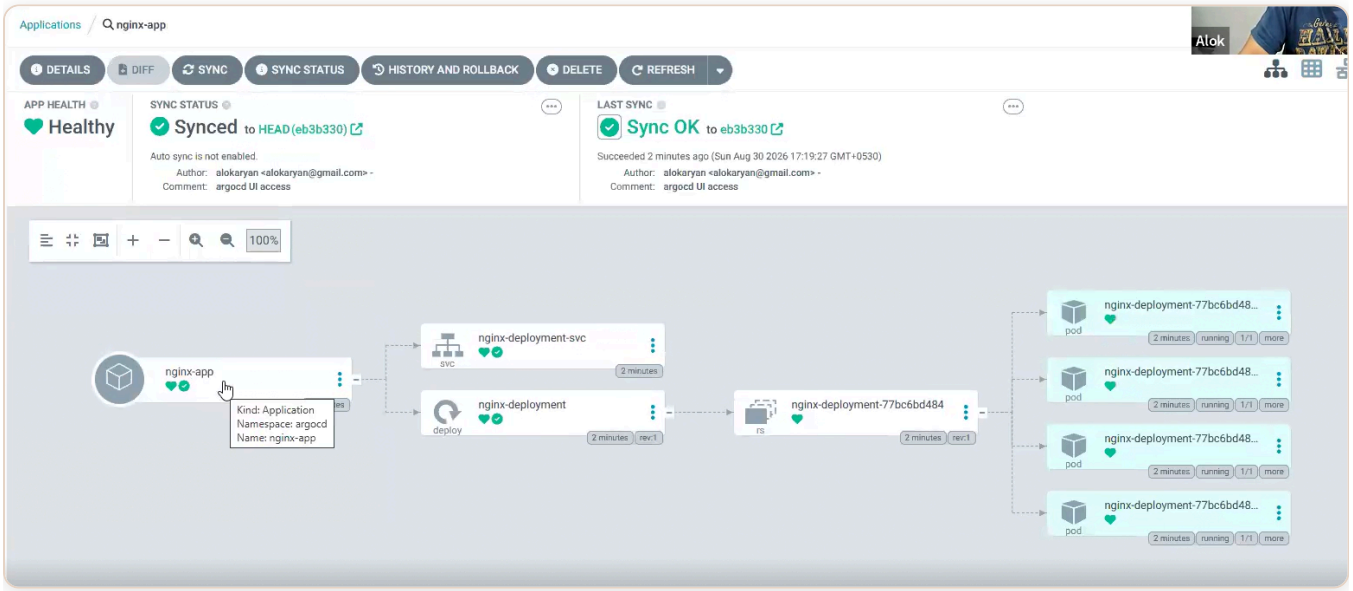
What Happens Right After Creation

The new app shows **Missing** and **OutOfSync** — expected, not an error. Sync Policy was Manual, so Argo CD deliberately applies nothing until you click **Sync**.

Step 3: Manually Sync

Click **Sync** → **Progressing** → **Synced**/Healthy.

```bash
kubectl get deploy
# the nginx deployment now appears - created BY Argo CD, sourced FROM Git
```



nginx-app resource tree — Healthy/Synced, Service + Deployment fanning out to 4 running pods

Part 2 — Deploying the Same App via the Argo CD CLI

The CLI was installed in Chapter 1. Now we authenticate it and use it to create the same kind of application imperatively.

Exposing Argo CD So the CLI Can Reach It

```bash
kubectl -n argocd get deploy      # confirm all 6 components are up first

kubectl -n argocd expose deployment argocd-server \
  --name argocd-server-np --type=NodePort
# service/argocd-server-np exposed

kubectl -n argocd get svc
# argocd-server-np   NodePort   10.96.102.186   <none>   8080:31530/TCP,8083:32517/TCP
#                                                                ^^^^^ this is the port to hit
```

Getting the Admin Password and Logging In via CLI

```bash
argocd admin initial-password -n argocd
# kzFS1lfzgwAVJdty
# This password must be only used for first time login. We strongly recommend you update
# the password using `argocd account update-password`.

argocd login 10.0.0.101:31530 --username admin --password kzFS1lfzgwAVJdty
# WARNING: server is not configured with TLS. Proceed (y/n)? y
# 'admin:login' logged in successfully
# Context '10.0.0.101:31530' updated
```

Basic CLI Checks, Then Creating an App Imperatively

```bash
argocd cluster list
# SERVER                          NAME        VERSION  STATUS      PROJECT
# https://kubernetes.default.svc  in-cluster  v1.35.8  Successful

argocd app list   # empty at first

argocd app create nginx-demo \
  --repo https://github.com/networknuts/argocd.git \
  --path ch-04 \
  --dest-server https://kubernetes.default.svc \
  --dest-namespace default
```

This is the **imperative** way of creating an application — same 3 required pieces as the UI form, just as CLI flags. Refreshing the UI shows the new app there too.

```bash
argocd app get nginx-demo
# Sync Status:  OutOfSync from  (eb3b330)
# Health Status: Missing

argocd app sync nginx-demo
# ...Service    default nginx-deployment-svc  Synced  Healthy
# ...Deployment default nginx-deployment      Synced  Healthy
```

Same `Missing` / `OutOfSync` → `Synced` / `Healthy` lifecycle as the UI path.

Downside of the imperative CLI approach

After some time you won't remember exactly what flags/values you used to create a given app — there's no file to refer back to. That's not the ideal way to manage applications long-term.

Part 3 — The Declarative Way: an Application YAML

Installing Argo CD registers a **custom resource** called `Application` — so you can define one the same declarative way as any other Kubernetes object, and keep that file in Git like everything else.

YAML

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: webapp
  namespace: argocd            # where Argo CD itself is installed
spec:
  project: default
  source:
    repoURL: 'https://github.com/networknuts/argocd.git'
    targetRevision: HEAD
    path: ch-04
  destination:
    server: 'https://kubernetes.default.svc'
    namespace: dev
  syncPolicy:
    automated:
      prune: true          # deletes resources removed from Git
      selfHeal: true       # overwrites manual in-cluster changes
```

BASH

```bash
kubectl apply -f my-argo-app.yaml
# application.argoproj.io/webapp created
```

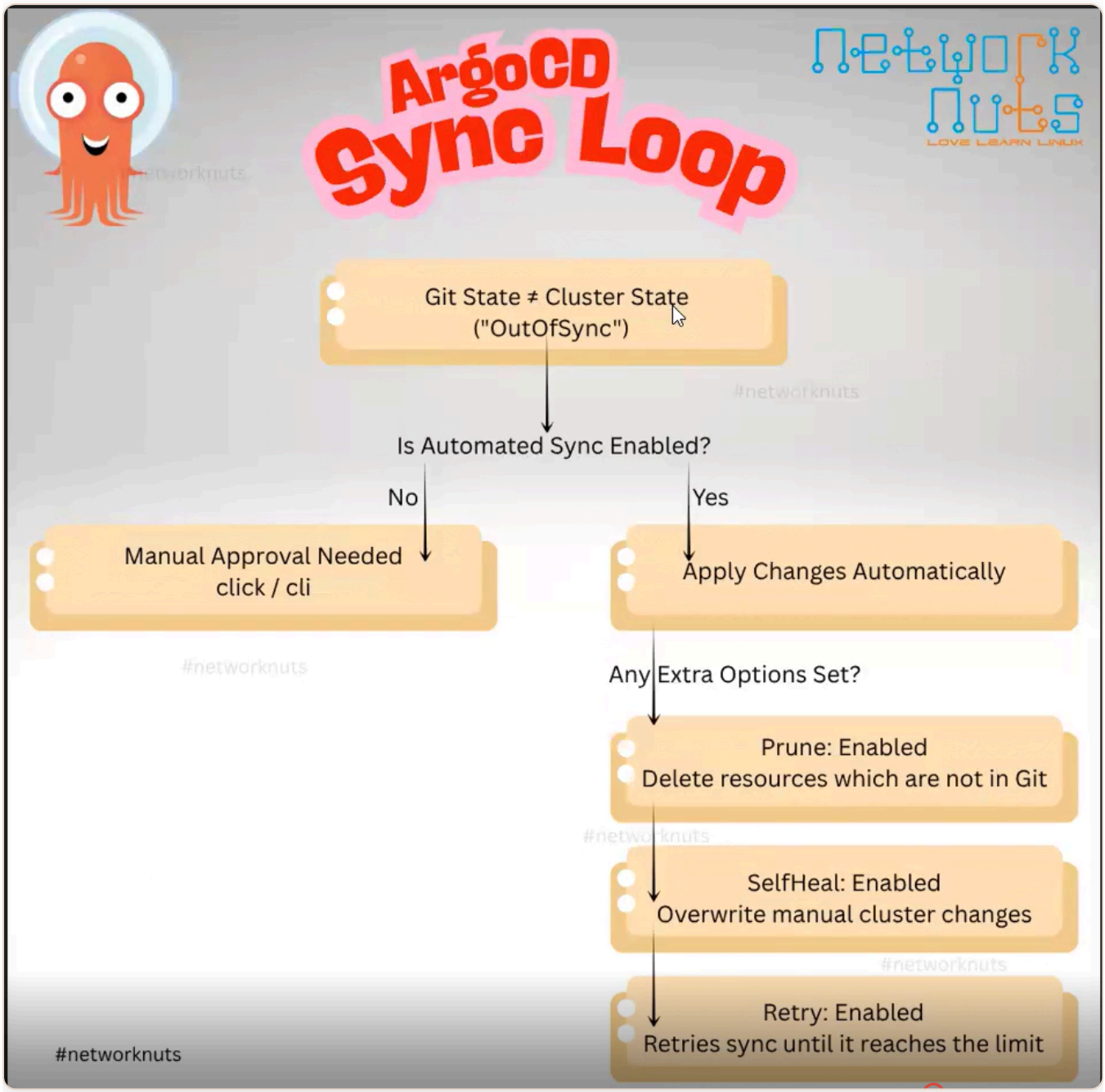
No Argo CD login needed for this path

You're talking to the Kubernetes API directly (already trusted via kubeconfig), not the Argo CD API server — the `Application` kind is just another custom resource at that point.

3 Ways to Create an Application

Method	Needs Argo CD username/password?	Style
Web UI	Yes	Imperative (click-driven)
`argocd` CLI	Yes (must `argocd login` first)	Imperative (command-driven)
`kubectl apply -f <Application-yaml>`	**No**	Declarative

Part 4 — The Argo CD Sync Loop, End to End

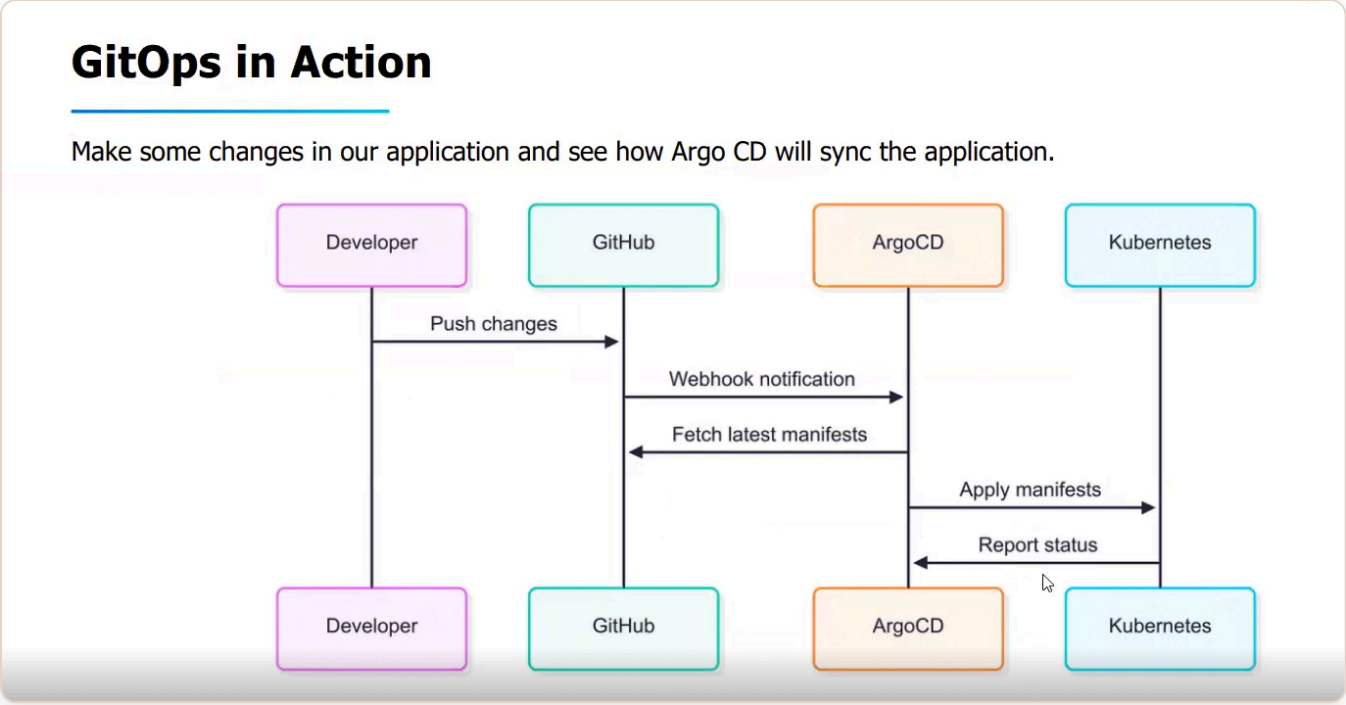


Argo CD Sync Loop: Git state != cluster state -> OutOfSync -> auto-sync decision -> prune / selfHeal / retry

1. **Git State ≠ Cluster State** → status becomes `OutOfSync`.
2. **Is Automated Sync enabled?**
   - **No** → Manual approval needed (click Sync in UI, or `argocd app sync` via CLI).
   - **Yes** → Argo CD applies changes automatically.
3. If automated, any extra options set take effect:
   - **Prune: Enabled** → deletes resources that exist in the cluster but are no longer in Git.

- **SelfHeal: Enabled** → overwrites manual changes made directly in the cluster (a `kubectl edit` on a managed object gets reverted back to match Git).
- **Retry: Enabled** → retries a failed sync until it hits its retry limit.

GitOps in Action — the Full Picture With a Webhook



Developer pushes to GitHub -> webhook notifies ArgoCD -> ArgoCD fetches manifests and applies them to Kubernetes -> reports status back

A webhook can replace/supplement the default polling for near-instant reaction to a push, instead of waiting up to the polling interval.

Part 5 — Troubleshooting Common Issues

Symptom	Commands to investigate
App stuck in `Progressing`	`argocd app get <app>`, `kubectl describe deployment <name>` — look for typos in manifests or image pull issues
Sync failures	`argocd app get <app>`, `kubectl logs -n argocd deployment/argocd-application-controller`
`OutOfSync` status, unclear why	`argocd app diff <app>` — compares Git vs. cluster state directly
Want a fresh comparison	`argocd app get <app> --refresh`

Part 6 — Deleting an Application, in Full

Deleting an Argo CD application sounds simple, but getting it wrong can either wipe out production workloads or leave orphaned resources cluttering the cluster. Two independent choices are involved.

Choice 1 — Cascade vs. Non-Cascade

- **Cascade delete** — Argo CD deletes the Application **and** every Kubernetes resource it manages (Deployments, Services, ConfigMaps, etc.).
- **Non-cascade delete** — Argo CD deletes only the Application resource itself, **leaving the managed Kubernetes resources running** in the cluster.

This behavior is controlled by a **finalizer** on the Application object:

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: webapp
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io   # enables CASCADE delete
spec:
  # ...
```

> **No finalizer line = default = non-cascade**
>
> Only the Application object is deleted; the Deployments/Services it created stay running.

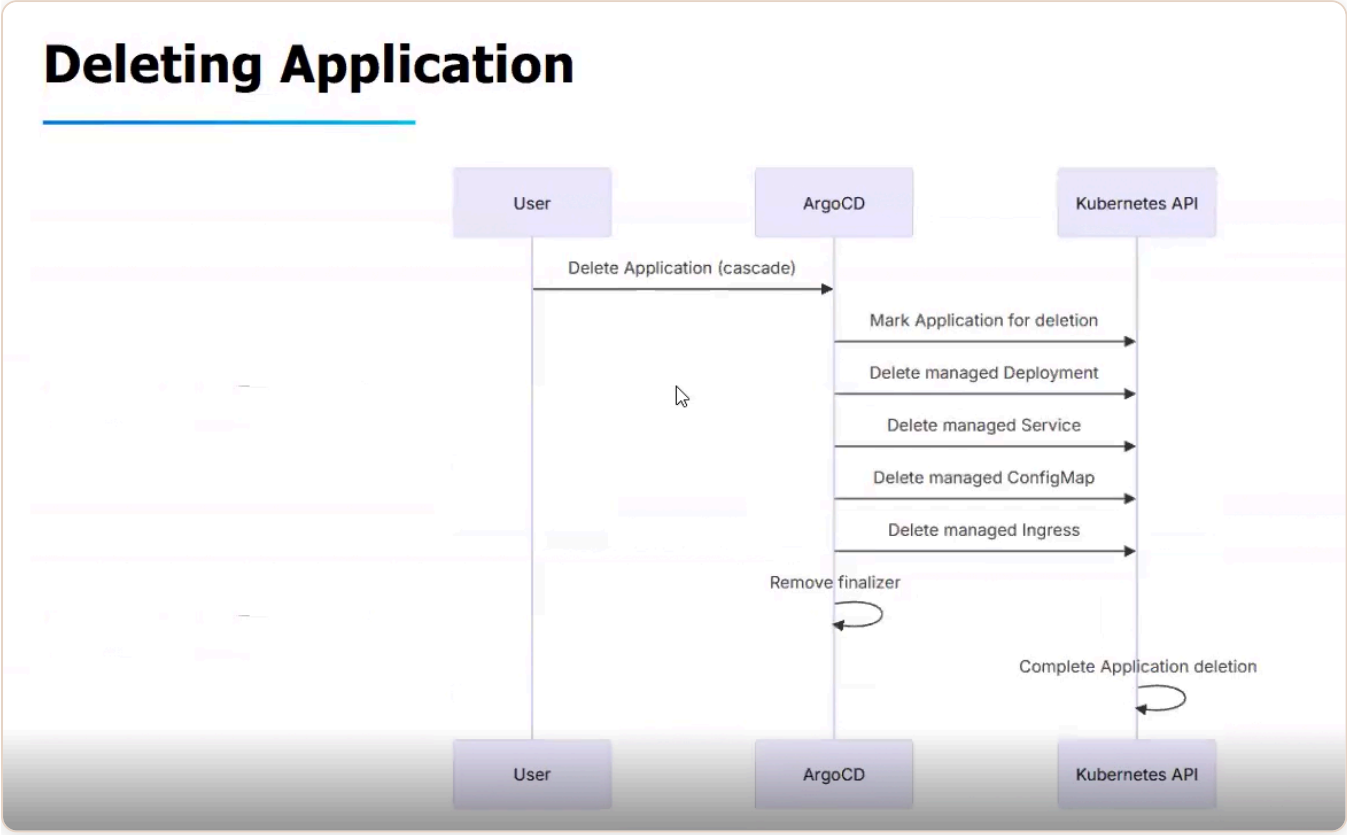
Choice 2 — Foreground vs. Background (Only Meaningful for Cascade Delete)

Mode	Behavior	Command
Foreground (default)	Argo CD uses standard Kubernetes garbage-collection logic. The root Application enters `Terminating` and **will not disappear** until every single managed resource (Deployments, Pods, Services) has been successfully deleted. Safer — ensures complete cleanup before the parent object goes away.	`argocd app delete APPNAME --cascade --propagation-policy foreground`
Background	The Application object is deleted **instantly**; the Kubernetes garbage collector then cleans up orphaned resources asynchronously behind the scenes. Faster, but carries a small risk that resource dependencies can stall in the background with no clear visibility in the Argo CD dashboard. **Not recommended** as a default choice.	`argocd app delete APPNAME --cascade --propagation-policy background`

How Cascade Delete Actually Works, Step by Step

1. Argo CD receives the delete request.

2. It checks for the `resources-finalizer.argocd.argoproj.io` finalizer on the Application.
3. If present, Argo CD identifies every resource it tracks for that application.
4. Argo CD deletes each managed resource from the cluster.
5. Once all resources are deleted (or deletion is initiated), Argo CD removes the finalizer.
6. Kubernetes then completes deletion of the Application resource itself.



Deleting Application sequence: User -> ArgoCD -> Kubernetes API -> mark for deletion, delete managed Deployment/Service/ConfigMap/Ingress, remove finalizer, complete deletion

The PVC caveat

Cascade delete will remove PersistentVolumeClaims if they are tracked/managed by Argo CD — your persistent data is genuinely at risk here. Before deleting, check:

```bash
argocd app resources app-name | grep PersistentVolumeClaim
```

If real data lives in a PVC that Argo CD manages, confirm you actually want it gone before running a cascade delete.

What I Practiced

- [ ] Connected a public Git repo to Argo CD via HTTPS in Settings → Repositories, confirmed "Successful" connection status.
- [ ] Created a new Application (`nginx-app` / `nginx-demo`) with Manual sync policy via both the UI and the `argocd` CLI, pointing at a specific branch + path, targeting the in-cluster destination and `default` namespace.
- [ ] Observed the `Missing` / `OutOfSync` status immediately after creation as the expected result of Manual sync — not a failure — via both interfaces.
- [ ] Clicked/ran Sync, watched the app move Progressing → Synced, and confirmed the resulting Deployment with `kubectl get deploy` .
- [ ] Exposed `argocd-server` as a NodePort from the bastion host, retrieved the initial admin password via `argocd admin initial-password` , and logged the CLI in against that NodePort.
- [ ] Wrote a declarative `Application` YAML with `syncPolicy.automated` (prune + selfHeal) and `kubectl apply -f` 'd it — confirmed it needed no separate Argo CD login, unlike the UI and CLI paths.
- [ ] Walked the Sync Loop decision tree end to end: OutOfSync → auto-sync check → prune/selfHeal/retry.
- [ ] Practiced the troubleshooting command set (`app get` , `app diff` , `app get --refresh` , `kubectl logs` on the application controller).
- [ ] Understood — flagged for hands-on testing next — the cascade/non-cascade and foreground/background delete distinctions, including the PVC-deletion risk.

Merged into the master reference notes: argocd-master-notes.md — expanded "2. First GitOps Application" section (CLI/declarative creation, sync loop, troubleshooting, deletion mechanics).